When Imbalance Correction Backfires: Robustness of Tabular Imbalance Mitigation Under Minority-Label Contamination

Sanjeev K. Khatri*, Zainab Albujasim, Youssef Harrath
The Beacom College of Computer and Cyber Sciences, Dakota State University
Madison, SD, USA
Emails: sanjeev.khatri@trojans.dsu.edu, Zainab.Albujasim@dsu.edu, youssef.harrath@dsu.edu
*Corresponding author: sanjeev.khatri@trojans.dsu.edu

Abstract—Imbalance-correction methods (resampling, cost-sensitive reweighting, and, for tabular foundation models, test-time posterior rescaling) are usually assumed to be safe. They might not always help much, but they are not supposed to hurt. That assumption is almost never tested under label noise. It is validated on clean data, then deployed into systems where labels are not clean. We test it directly, using TabPFN [1], [2] (threshold correction, DistPFN [3], DistPFN-T, class-balanced downsampling) and XGBoost [4] (cost-sensitive weighting, random oversampling, SMOTE [5]), we inject two directed, asymmetric label-error mechanisms (majority rows mislabeled as minority, and minority rows lost into the majority class) across 26 UCI datasets, at four imbalance ratios and five contamination rates, holding total training-set size fixed so that changes in class ratio are not confounded by changes in the amount of training data. Across 112,319 successfully completed model-condition evaluations (one additional evaluation failed to converge numerically and is reported separately), all seven corrections underperformed the uncorrected baseline in at least some tested conditions, with correction-harm rates ranging from 2.4% to 47.4%, depending on the method and corruption direction. A robustness check isolating the balanced (r=0.50) control condition from the genuinely imbalanced ratios shows the upper end of that range falls to 37.8% once restricted to actual imbalance, while the most robust method's advantage is unaffected or strengthened. Under some dataset-imbalance settings, corrections reached their break-even point at only 5% contamination. Correction-method performance differed significantly within both model families under both corruption directions in the conservative, dataset-level Friedman analysis (all four omnibus tests, $p < 10^{-9}$). No correction remained non-degrading across all tested conditions in both corruption directions. If label quality cannot be guaranteed, and in practice it rarely can be, imbalance correction should not be applied on the assumption that it is harmless.

Index Terms—class imbalance, label noise, imbalance correction, TabPFN, XGBoost, calibration

I. Introduction

Class imbalance arises when one class is substantially less frequent than the other, often degrading a model's ability to recognize the minority class. Under severe skew, a classifier may achieve deceptively high overall accuracy while exhibiting poor minority-class recall, because majority predictions dominate the evaluation.

To address this, practitioners commonly turn to three kinds of correction. Data-level resampling techniques, such as SMOTE [5] and random oversampling, add or duplicate minority examples before training begins. Cost-sensitive reweighting [6] instead leaves the data untouched and penalizes mistakes on the minority class more heavily during training. A third kind, threshold and posterior-probability correction such as DistPFN [3], is suited to tabular foundation models such as TabPFN [1], [2]. Instead of touching the data or the training process, it adjusts the model's predictions after they have already been made.

Many imbalance-correction strategies are evaluated under an implicit condition: that the labels used to identify and characterize the minority class are sufficiently reliable. Although class imbalance and label noise have each been studied extensively, considerably less is known about whether standard imbalance corrections remain beneficial when minority-label information is systematically corrupted (Section II).

If that assumption does not hold, none of these corrections can tell a genuine minority example from a mislabeled one. A resampling method will duplicate or synthesize around a mislabeled row exactly as readily as a correctly labeled one. A cost-sensitive method will derive its penalty from the observed class ratio, so any corruption in that ratio carries directly into the penalty. A posterior-rescaling method will correct a prediction against an observed class prior that carries the same corruption. In every case, the correction can end up amplifying the mislabeled examples it was built to compensate for, rather than correcting for genuine class imbalance.

This paper tests that failure mode directly, measuring when standard imbalance-correction methods help and when they backfire under controlled, directed label contamination. We organize this investigation around four research questions.

- RQ1. Do standard imbalance corrections remain beneficial once directed label error is present?
- RQ2. How do correction failure rates differ between the two directed corruption mechanisms?
- RQ3. At what contamination rate does each correction stop paying off?

- RQ4. How much of a correction's failure comes from using a corrupted prior instead of the true, oracle prior?

Concretely, we audit seven correction methods across two widely deployed tabular model families, TabPFN and XGBoost [4], under two directed, asymmetric label-error mechanisms and a range of imbalance levels, using 26 real datasets drawn from the UCI Machine Learning Repository [7]. The design holds the total amount of training evidence fixed across imbalance levels, so that changes in imbalance are not confounded by changes in how much training evidence a method receives. Section III gives the full experimental setup, including the exact corruption procedure and every configuration tested.

To answer these questions, we judge whether a correction helps or hurts, rather than only how accurate it is, using five purpose-built metrics. Correction Gain and Correction Harm Rate answer RQ1 and RQ2, measuring whether and how often a correction underperforms its uncorrected baseline. The break-even contamination rate answers RQ3, and the Prior Sensitivity Gap answers RQ4. A fifth metric, Regret, does not map onto a single research question. It measures how severe a failure is when one occurs, complementing Correction Harm Rate's count of how often one does. Each metric is defined precisely in Section III.

Our central finding is that every correction method tested underperforms its matched uncorrected baseline in at least some conditions. How often this happens depends strongly on both the direction of the corruption and the dataset the correction is applied to, and no correction remains non-degrading across every tested condition in both corruption directions. The complete results, including per-method failure rates and break-even points, are reported in Section IV.

Our contributions are as follows.

- We introduce a controlled experimental design that holds total training-set size fixed across imbalance ratios, so that changes in training-set size are not confounded with changes in class imbalance, and injects two directed, asymmetric label-error mechanisms at every tested ratio.
- We audit seven correction methods across two widely used model families, including DistPFN and DistPFN-T [3]. We validate our DistPFN and DistPFN-T implementation against the authors' published reference code and document an implementation error identified during this verification process (Section III).
- We operationalize correction robustness using five complementary measures, Correction Gain, Correction Harm Rate, Regret, break-even contamination rate, and Prior Sensitivity Gap, designed to measure whether a correction remains net-beneficial once label noise is present, rather than measuring model accuracy alone.
- We show, across 26 real datasets, that every correction method tested can underperform doing nothing at all once contamination reaches as little as 5%, and that this failure is direction-dependent rather than uniform across methods or datasets.

II. Related Work

A. Tabular Foundation Models and In-Context Learning

TabPFN [1] introduced a transformer pretrained once on millions of synthetic datasets that performs supervised tabular classification through in-context learning, the training set is presented as a prompt, and a single forward pass yields predictions for the entire test set, without any gradient-based fitting on the target data. The follow-up TabPFN v2 release [2] scaled the pretraining prior and architecture, reporting accuracy that matches or exceeds tuned gradient-boosted ensembles on datasets of up to ten thousand rows while running orders of magnitude faster. A concurrent empirical comparison across classical, deep-learning, and tabular foundation models on imbalanced clinical data [8] found that TabPFN v2 and a related in-context model, TabICL, rank competitively with tuned gradient-boosted ensembles on one imbalanced clinical dataset but fall to intermediate, less stable ranks on another, with XGBoost the only model family stable across both. That comparison, like the TabPFN v2 release itself, evaluates imbalance robustness only under clean, uncorrupted labels, leaving open whether either family's apparent robustness to imbalance survives once label quality is also compromised, the question this paper asks directly. Because a Prior-Data Fitted Network's predictions are shaped entirely by the class distribution implicit in its pretraining prior together with the context it is given, its behavior under a training-time class distribution that differs from what the prior assumes is a distinct concern from that of a conventionally trained classifier, and has only recently begun to receive dedicated attention. Hu and Ghelichi [9] take a complementary approach, analyzing TabPFN's internal attention mechanisms directly under synthetic label-quality degradation and finding that its attention stays structured and its ROC-AUC stays high without any correction applied at all. Their analysis targets the model's internals rather than the correction methods practitioners actually deploy on top of it, the gap this paper's audit addresses.

DistPFN [3] is the newest instance of a much older idea, adjusting a classifier's output probabilities to match a known or assumed target class prior, without retraining, first formalized for conventional probabilistic classifiers by Saerens et al. [10]. To our knowledge, DistPFN is the first method to bring this idea to an in-context tabular foundation model, where the "prior" being adjusted for is implicit in the pretrained transformer rather than an explicit model parameter: it rescales TabPFN's predicted class probabilities to counteract label shift between the training context and the deployment distribution, with

a temperature-scaled variant (DistPFN-T) that adapts the adjustment strength to the discrepancy between the observed and reference priors. Both variants require no retraining or architectural change, which places them in the same class-imbalance setting that classical resampling and cost-sensitive techniques were designed for on conventional classifiers, despite operating through a completely different mechanism.

B. Correcting Class Imbalance

The dominant strategies for class imbalance fall into three families: data-level resampling, algorithm-level cost adjustment, and, for in-context tabular models, the test-time posterior corrections described above. SMOTE [5] synthesizes new minority-class examples by interpolating between real minority points and their nearest minority neighbors in feature space, rather than duplicating existing rows, and remains one of the most widely adopted correction techniques nearly a quarter-century after its introduction. Random oversampling, by contrast, corrects the training distribution by duplicating existing minority rows without synthesizing new feature vectors. Both are implemented as standard, interchangeable components in imbalanced-learn [11], a widely used toolbox implementing both corrections. Cost-sensitive learning [6] instead leaves the training distribution untouched and reweights the loss so that minority-class errors are penalized more heavily than majority-class errors in proportion to the observed class ratio. XGBoost's scale_pos_weight parameter [4] is a direct, widely used implementation of this idea. Whether these corrections reliably help at all, even when every label is trustworthy, is itself contested: Elor and Averbuch-Elor [12] find that with a sufficiently strong classifier and well-chosen hyperparameters, balancing the training set often provides no benefit over the imbalanced data as-is, and can be actively counterproductive, a finding that already complicates the assumption that these corrections are safe by default, before label noise is introduced at all. All of these methods share a further, implicit assumption that is central to this paper, that the observed class labels used to compute the resampling ratio, the interpolation neighborhood, or the cost ratio are themselves trustworthy. These classical corrections were not originally designed for robustness to label noise. Subsequent empirical work, discussed later in this section, has evaluated several sampling approaches under noisy-label settings, but considerably less is known about the newer test-time posterior corrections built for tabular foundation models.

C. Learning Under Label Noise

A separate literature studies classification when the observed labels themselves are corrupted, independently of any class imbalance. Early theoretical work formalized class-conditional label noise, in which the probability that a true label is flipped depends only on its true class, and established conditions under which consistent classifiers can still be recovered [13]. Asymmetric label noise, where the flip probabilities differ across classes or flip in only one direction, was shown to require different identifiability and denoising strategies than the symmetric case [14], indicating that the direction and structure of label corruption, not only its rate, can materially change how a classifier should be corrected. Frénay and Verleysen [15] survey sources and taxonomies of label noise and its effects on classifier robustness prior to the deep learning era, while Song et al. [16] survey the substantially larger body of work on training deep networks to be robust to noisy labels, including loss correction, sample selection, and label correction approaches. Both surveys treat label noise as a phenomenon to be corrected or tolerated during training. Neither the surveys nor the methods they catalog examine how label noise interacts with a class-imbalance correction applied alongside it, which is the specific interaction this paper studies, not by proposing a new noise-robust training method, but by auditing whether existing, widely deployed imbalance corrections remain net-beneficial once that assumption is violated.

D. The Understudied Intersection: Imbalance Correction Under Noisy Labels

Empirically auditing whether imbalance corrections hold up under label noise has important precedent. Van Hulse and Khoshgoftaar [17] trained more than two million classifiers across seven datasets, eleven learners, and seven sampling techniques, including random oversampling and SMOTE, while varying the location and level of class noise, showing that the class receiving the noise materially affects how learning and sampling procedures behave and establishing that imbalance mitigation can interact strongly with label noise in classical learning settings. Our study addresses a complementary setting, We control the total training-set size across imbalance ratios so that changes in training-set size are not confounded with changes in class imbalance, and we extend the analysis to tabular foundation models and recently proposed test-time posterior corrections such as DistPFN and DistPFN-T.

A smaller body of work has begun to treat class imbalance and label noise as jointly occurring problems rather than independent ones. Koziarski et al. [18] combine a resampling step with a label-cleaning step specifically for multi-class imbalanced data known to contain label noise. Chen et al. [19] and Li et al. [20] take a related but more direct approach, redesigning SMOTE itself so that its neighborhood-based interpolation is weighted by local sample density and downweights or filters out likely-noisy minority points, rather than pairing an unmodified SMOTE with a separate cleaning stage. Khanal et al. [21] combine active learning with noisy-label training to handle both problems together in imbalanced medical image classification. Liu et al. [22] observe that standard loss-based

sample selection for noisy-label training is itself biased against minority classes, since under-learned tail classes and noise-corrupted classes are both associated with high training loss and are consequently difficult to tell apart. They propose a class-balanced sample selection procedure with confidence-based augmentation to correct this bias during training. A recent line of work inverts the relationship between the two problems entirely: Hu et al. [23] show that deliberately injecting an asymmetric label noise model, engineered specifically to counteract a classifier's biased decision boundary, can rebalance an imbalanced classifier without resampling at all. Their result and ours are not in tension. They show that noise designed in advance to help, with its direction and magnitude chosen to correct a known bias, can substitute for a resampling or reweighting step; we show that the standard resampling and reweighting steps already in deployment cannot tell that engineered noise apart from the generic, uncontrolled label errors a real dataset actually contains, and so extend the same corruption-tolerance assumption to the corrections themselves rather than to the raw classifier. Each of these lines of work proposes a new algorithm for the training-time mechanism of handling noisy labels under imbalance. None of them evaluates whether the standard imbalance corrections already in widespread use (SMOTE, random oversampling, cost-sensitive reweighting, or, for the newer class of tabular foundation models, test-time posterior adjustment and class-balanced downsampling) continue to help, are neutral, or actively harm performance once a controlled fraction of the labels they operate on are themselves wrong. What remains open, to our knowledge, is whether these corrections, applied to tabular foundation models under a directed, asymmetric label-error rate, still improve on doing nothing at all rather than merely being neutral or harmful. Closing that gap is what this paper's controlled, cross-method audit is designed to do.

III. Methods

This section describes the full experimental design: the datasets, the construction that holds training-set size fixed across imbalance ratios, the two directed label-error mechanisms, the seven correction methods audited, the metrics used to score them, and the statistical tests used to compare them.

A. Overview

Figure 1 summarizes the pipeline applied to every dataset. A dataset is split once into a training pool and a held-out test set, which stays fixed and clean for the rest of that seed's run. A controlled sample is drawn from the training pool at a target minority ratio r and a fixed total size N_{common} (Section III-C). A directed label error is then injected into that sample at a contamination rate c, in one of two directions (Section III-D), or left clean at c=0. Every correction method (Section III-E) is fit on the resulting training set and scored against the same held-out test set, and the scores are converted into the metrics described in Sections III-F and III-G. This is repeated for every combination of dataset, seed, imbalance ratio, contamination rate, and direction, and for every correction method, producing one row per method per condition.

B. Datasets

Twenty-six binary-classification datasets were drawn from the UCI Machine Learning Repository [7]: Banknote Authentication [24], Blood Transfusion [25], South German Credit [26], Spambase [27], Breast Cancer Wisconsin [28], Ionosphere [29], Haberman's Survival [30], Statlog Heart [31], HTRU2 [32], QSAR Biodegradation [33], Adult [34], Bank Marketing [35], MAGIC Gamma Telescope [36], Online Shoppers Purchasing Intention [37], Congressional Voting Records [38], Chronic Kidney Disease [39], Default of Credit Card Clients [40], Taiwanese Bankruptcy Prediction [41], Polish Companies Bankruptcy [42], Mushroom [43], Cervical Cancer (Risk Factors) [44], SECOM [45], Arrhythmia [46], Internet Advertisements [47], Seismic-Bumps [48], and APS Failure at Scania Trucks [49], chosen so that no single result could be attributed to one dataset's idiosyncrasies, spanning finance, security, healthcare, signal processing, astronomy, chemistry, industrial manufacturing, retail behavior, and political science. Together they range from 270 to 60,000 raw rows and from 3 to 1,558 raw features, and their natural minority prevalence ranges from 1.7% (APS Failure) to 62.5% (Chronic Kidney Disease, the one dataset in this study where the clinically "positive" class is the natural majority). Table I summarizes all 26 datasets' size, the controlled training size N_{common} every stress cell actually draws from, and the meaning of the positive (minority) class in each.

No transformation altered the semantics of any feature. Two categories of transformation changed feature values or feature counts, neither altering what a feature means. First, column renames, purely cosmetic, applied to Spambase and QSAR Biodegradation: XGBoost's tree serialization format rejects the characters [,], and < in feature names, so Spambase's char_freq_[(the frequency of the [character in an email) was renamed to char_freq_bracket_open, and eight of QSAR Biodegradation's molecular-descriptor columns (e.g., F01[N-N]) had their brackets replaced with a hyphen (e.g., F01_N-N), with the underlying values unchanged in both cases. Second, a feature cap applied to the four datasets with more than 100 raw features, SECOM (590), Arrhythmia (279), Internet Advertisements (1,558), and APS Failure (170): each is capped to a deterministic random subsample of 100 columns (fixed seed, sorted for a stable, reviewable column order), so that no method's training cost scales with raw feature count rather than with the imbalance and noise conditions actually under test. The columns

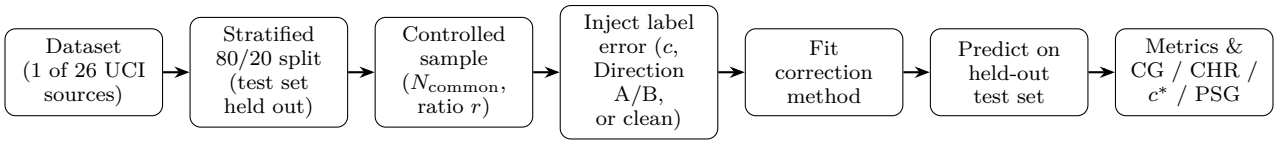


Fig. 1. The stress-cell pipeline applied to every dataset, seed, imbalance ratio, contamination rate, and direction. The test set is held out once, at the initial split, and stays fixed and clean for the entire run. Only the training side is resampled and corrupted before each correction method is fit and then scored against that same held-out test set.

TABLE I

Dataset characteristics, all 26 datasets. N_{common} (Section III-C) is the controlled training size every stress cell actually draws from, not the raw row count, computed at seed 13; seed-to-seed variation is at most a few rows due to stratified-split rounding. Feature counts marked →100 are capped (Section III-B).

Dataset	Rows	Feat.	N_{common}	Minority meaning
Banknote Authentication	1,372	4	641	Forged banknote
Blood Transfusion	748	4	284	Blood donated in target period
South German Credit	1,000	20	480	Bad credit risk
Spambase	4,601	57	2,347	Spam email
Breast Cancer Wisconsin	569	30	300	Malignant diagnosis
Ionosphere	351	34	189	No ionospheric structure
Haberman's Survival	306	3	128	Death within 5 years
Statlog Heart	270	13	126	Heart disease present
HTRU2	17,898	8	2,622	Pulsar candidate
QSAR Biodegradation	1,055	41	568	Ready biodegradable
Adult	48,842	14	18,698	Income exceeds \$50K/year
Bank Marketing	45,211	16	8,462	Client subscribed to term deposit
MAGIC Gamma Telescope	19,020	10	10,385	Hadron (background) event
Online Shoppers	12,330	17	3,052	Session ended in completed purchase
Congressional Voting	435	16	225	Republican representative
Chronic Kidney Disease	400	24	126	Chronic kidney disease present
Default of Credit Card Clients	30,000	23	10,618	Default on payment next month
Taiwanese Bankruptcy	6,819	95	352	Company went bankrupt
Polish Companies Bankruptcy	43,405	65	3,346	Bankrupt within forecast period
Mushroom	8,124	22	3,543	Poisonous mushroom
Cervical Cancer (Risk Factors)	858	32	88	Biopsy-confirmed abnormality
SECOM	1,567	590→100	166	Semiconductor test fail
Arrhythmia	452	279→100	206	Cardiac arrhythmia present
Internet Advertisements	3,279	1,558→100	734	Image is an advertisement
Seismic-Bumps	2,584	18	272	Hazardous seismic state next shift
APS Failure at Scania Trucks	60,000	170→100	1,600	APS component failure

kept are fixed once per dataset and identical across every seed, ratio, and contamination level tested on it.

The pipeline separately includes leakage-safe median imputation for numeric columns and most-frequent-value imputation followed by one-hot encoding for categorical columns, fit only on each stress condition's training rows. Ten of the 26 datasets, Banknote Authentication, Blood Transfusion, South German Credit, Spambase, Breast Cancer Wisconsin, Ionosphere, Haberman's Survival, Statlog Heart, HTRU2, and QSAR Biodegradation, contain no missing values and no non-numeric feature column (South German Credit's twenty attributes and Statlog Heart's categorical clinical measures are integer-coded and are treated as numeric throughout), so neither step alters any value for those ten. Of the remaining sixteen, eleven contain at least one column with missing values (Adult, Bank Marketing, Congressional Voting, Chronic Kidney Disease, Polish Companies Bankruptcy, Mushroom, Cervical Cancer, SECOM, Arrhythmia, Internet Advertisements, and APS Failure) and seven contain genuinely categorical, non-numeric columns (Adult, Bank Marketing, Online Shoppers, Congressional Voting, Chronic Kidney Disease, Mushroom, and Seismic-Bumps), so both imputation steps and the one-hot encoding step are genuinely exercised in this study, rather than being present in the pipeline but never triggered.

Before any sampling or corruption, each dataset is split once with a stratified 80/20 train/test split, so both partitions preserve the dataset's original class ratio. The test partition is fixed for that seed and never altered by any later step. Every correction method, at every imbalance ratio and contamination level, is scored against exactly the same test rows.

That fixed test partition has a consequence worth naming directly. Because it is never resampled, it keeps each dataset's natural minority prevalence throughout, while Section III-C deliberately reshapes the training partition to r. Natural prevalence across the 26 datasets ranges from 1.7% (APS Failure) to 62.5% (Chronic Kidney Disease). Seven datasets sit at or below the two most

severe tested ratios: APS Failure (1.7%) and Taiwanese Bankruptcy (3.2%) fall below even r=0.05; Polish Companies Bankruptcy (4.8%) sits almost exactly on it; and Cervical Cancer (6.4%), Seismic-Bumps (6.6%), SECOM (6.6%), and HTRU2 (9.2%) all sit close to r=0.10. So at r=0.05 and r=0.10 especially, training and test are drawn from different class priors for a substantial share of the datasets in this study. The severe-imbalance conditions this study emphasizes are therefore not imbalance in isolation. They combine imbalance with a train/test prior shift, exactly the kind of mismatch DistPFN and DistPFN-T are built to correct (Section III-E), and the kind tabpfn_threshold's own threshold adjustment addresses as well. Section IV-B returns to what this means for interpreting why the simplest corrections in this study lead, and Section V-A states the resulting limitation on treating imbalance and prior shift as separable.

C. Controlled Imbalance: A Common Training Size

Each dataset is tested at four minority ratios, $r \in \{0.50, 0.20, 0.10, 0.05\}$, where r is the fraction of training rows that belong to the minority class. Comparing a model trained at a high ratio (e.g., r=0.20) against one trained at a low ratio (e.g., r=0.05) is only informative about imbalance itself if both training sets are the same total size. Otherwise, a difference in performance could be explained by sample size rather than by imbalance. To remove that confound, one shared training size N_{common} is computed for each dataset and seed and used at every tested ratio. Only the split between minority and majority rows within that fixed total changes as r changes.

Let $n_{\min}$ and $n_{\max}$ denote the number of minority and majority rows available in the training pool for a given dataset and seed. At ratio r, a total training size of N requires rN minority rows and $(1-r)N$ majority rows, so the largest N achievable at that ratio is bounded by whichever class runs out first:

$$\text{feasible_n}(r) = \min\left(\frac{n_{\min}}{r},\ \frac{n_{\max}}{1-r}\right). \tag{1}$$

N_{common} is the largest size achievable at every tested ratio simultaneously, i.e., the tightest of these four constraints, floored to a whole number of rows:

$$N_{\text{common}} = \left\lfloor \min_{r\in\{0.50,0.20,0.10,0.05\}} \text{feasible_n}(r) \right\rfloor. \tag{2}$$

For a given r and N_{common}, a controlled training set is drawn by sampling $\text{round}(r \cdot N_{\text{common}})$ minority rows and the remaining N_{common} rows from the majority class, both without replacement, then shuffling the combined set.

D. Directed Label-Error Mechanisms

Two label-error mechanisms are applied to the controlled training set built in Section III-C, each at contamination rate $c \in \{0.05, 0.10, 0.20, 0.30\}$, plus a shared clean condition at c=0. Both flip a controlled number of true labels within that set, but they differ in which class is flipped and how c is defined relative to the resulting pool. Here $n_{\min}$ denotes the number of true minority rows in that controlled training set, equal to $\text{round}(r \cdot N_{\text{common}})$ by construction, not the pool-level $n_{\min}$ from Section III-C.

Direction A (contamination): majority rows relabeled as minority. c is defined as the fraction of the observed minority-labeled pool that is actually mislabeled majority rows. If k majority rows are flipped,

$$c = \frac{k}{n_{\min}+k} \quad\Longrightarrow\quad k = \frac{c}{1-c}\, n_{\min}. \tag{3}$$

k rows are drawn uniformly at random from the majority pool and relabeled.

Direction B (loss): minority rows relabeled as majority. Here c is defined directly against the true minority count:

$$k = c \cdot n_{\min}. \tag{4}$$

k rows are drawn uniformly at random from the minority pool and relabeled.

Both mechanisms return the corrupted labels together with the realized contamination rate (which can differ slightly from the requested c due to integer rounding of k) and the indices flipped, so every corruption is fully auditable. This asymmetric, class-conditional construction follows the label-noise formalization of Natarajan et al. [13] and Scott et al. [14], rather than symmetric random flipping.

That difference in definition is not cosmetic. Direction A's c describes contamination as it would be observed after the fact, the fraction of everything now labeled minority that is actually a mislabeled majority row, close to asking how many flagged cases turn out to be false alarms. Direction B's c instead measures signal loss directly against the true minority count, the fraction of genuine minority rows destroyed. The two are natural for the process each direction represents, but they are not the same quantity. The same nominal c therefore corresponds to a different flip count relative to $n_{\min}$ in each direction. At c=0.30, Direction A flips $k/n_{\min} = c/(1-c) = 42.9\%$ of $n_{\min}$'s worth of majority rows, while Direction B flips exactly 30%, a gap that widens with c (from 5.3% against 5.0% at c=0.05) but stays independent of r, since both ratios are taken relative to $n_{\min}$ within their own direction's formula rather than against the majority count. Comparing directions at matched nominal c, as Section IV-D does, is therefore a comparison of each direction's own natural severity, not of an identical absolute flip count, a distinction Section V-A returns to.

E. Correction Methods

Seven correction methods are audited, split into two families so that every method is compared against the uncorrected baseline for the same underlying model. All XGBoost variants share one frozen configuration (Table II), never tuned per condition, so any performance difference between them is attributable to the correction technique itself.

1) XGBoost family: xgb_none trains XGBoost [4] directly on the corrupted, imbalanced training set with no correction, and is the baseline every other XGBoost variant is measured against. xgb_weighted sets scale_pos_weight to the ratio of observed majority to observed minority rows, following the cost-sensitive framing of Elkan [6]. xgb_ros applies random oversampling, duplicating minority-labeled rows with replacement until the classes are balanced. xgb_smote instead synthesizes new minority-labeled rows with SMOTE [5], interpolating between a minority row and one of its k nearest minority neighbors, a parameter unrelated to the flip count k in Section III-D. It is set to $k = \min(5, n_{\min}^{\text{obs}} - 1)$, where $n_{\min}^{\text{obs}}$ is the observed minority count in the corrupted training set actually being resampled (distinct from both $n_{\min}$ in Section III-C and $n_{\min}$ in Section III-D), so the request never exceeds the neighbors actually available. Both resampling variants use imbalanced-learn [11].

2) TabPFN family: TabPFN [1], [2] is an in-context tabular foundation model: it is not fit with gradient descent on the study's data, but reads the entire training set as a prompt and produces a prediction for each test row in a single forward pass. Every TabPFN variant uses the library's default hyperparameters, with only the random seed set per run, so no TabPFN-specific configuration table is needed alongside Table II. tabpfn_none takes its raw $P(\text{minority})$ and thresholds at 0.5, and is the baseline for the rest of the family. tabpfn_threshold instead thresholds at π, the class-1 prior used for correction, following the classical posterior-adjustment principle of Saerens et al. [10]: predict minority if $P(\text{minority}) > \pi$. As with every prior-dependent method in this family, π is either the observed, possibly corrupted training rate π_{obs} (the deployable variant) or the true pre-corruption rate π_{true} (the oracle diagnostic described in Section III-G).

tabpfn_distpfn and tabpfn_distpfn_t implement DistPFN and DistPFN-T [3]. Writing p for TabPFN's raw 2-class prediction on one test row and $\bar{p}$ for the mean of p across the whole test batch,

$$\text{prior} = [1 - \pi,\ \pi], \qquad \text{adjusted} = \text{Normalize}\left(\frac{p \cdot \bar{p}}{\text{prior}}\right), \tag{5}$$

using the same class-1 prior π (observed or oracle, as defined above). DistPFN-T adds a shared temperature τ before the same final step:

$$\tau = -\sum_i \bar{p}_i \log(\text{prior}_i), \qquad \bar{p}_T = \text{softmax}(\bar{p}/\tau), \tag{6}$$

with $\bar{p}_T$ substituted for $\bar{p}$ in Equation 5. Both formulas were verified directly against the authors' official reference implementation before use in this study. That verification surfaced a defect in the reference implementation itself, reported here rather than left implicit: the authors' own published DistPFN-T reference driver raises a shape error (numpy's AxisError) for binary classification, which its own exception handling silently swallows, so the reference driver as published cannot produce a DistPFN-T score for a two-class problem. Equation 6 implements the algorithm's evident intent, with that shape bug corrected rather than reproduced.

tabpfn_downsample is the family's one data-level correction: whichever class has more observed rows is randomly downsampled, without replacement, to match the observed count of the smaller class, and TabPFN reads the resulting balanced set as its prompt. Identifying the larger class from the observed labels directly, rather than assuming a fixed encoding stays the larger one, means this never fails regardless of how contamination shifts the observed class balance, including at r=0.50 under Direction A, where relabeling majority rows as minority can push the observed minority count above the observed majority count.

For prior-dependent methods (tabpfn_threshold, tabpfn_distpfn, tabpfn_distpfn_t), every stress cell (one dataset, seed, ratio, contamination rate, and direction) is scored twice: once using the observed prior π_{obs}, the only value available in a real deployment, and once using the oracle prior π_{true}, the clean pre-corruption rate, which a real system could never know. The oracle variant exists only to compute the Prior Sensitivity Gap (Section III-G) and is never treated as a deployable result.

F. Evaluation Metrics

Every method, on every stress cell, is scored with ten metrics computed against the fixed, clean test set: balanced accuracy, worst-class recall (the lower of the two per-class recalls, which catches a collapse in one class that balanced accuracy alone can hide), area under the precision-recall curve (AUPRC), macro-averaged F1, minority-class precision and recall, area under the ROC curve (AUROC), Brier score [50], log loss, and expected calibration error (ECE, ten equal-width bins) [51], which measures how far a model's predicted probabilities are from the true observed frequencies.

All ten are computed identically for every method and condition using scikit-learn's implementations [52], so no method is scored on a different basis than any other.

Not every metric receives equal treatment in Section IV. Balanced accuracy is reported as the discrimination measure throughout. Worst-class recall, AUPRC, and macro F1 correlate with it at 0.968, 0.781, and 0.919 respectively (Pearson, computed across the full result set) and largely reproduce the same ranking rather than adding independent evidence. AUPRC's correlation is the loosest of the three, consistent with this study's wide range of natural imbalance producing real divergence between the two metrics, but not loose enough to promote it from a secondary diagnostic to a headline measure on its own. Minority precision, minority recall, and AUROC are computed as secondary diagnostics, available for a targeted check but not used to support a headline claim on their own. Brier score and expected calibration error are

reported separately in Section IV-J because together they reveal a divergence from the discrimination results that balanced accuracy alone does not show (Section IV-K). Log loss is computed identically to the other nine metrics but is not separately reported, since its correlation with Brier score, 0.959, is comparably high and does not add evidence beyond it.

G. Contribution Metrics

The ten metrics above measure how good a method is in isolation. The following five metrics answer the question this study is actually about: whether a correction is worth applying once label noise is present, relative to doing nothing at all. All five compare a corrected method against its matching uncorrected baseline (xgb_none or tabpfn_none) at the same dataset, seed, ratio, contamination rate, and direction.

Correction Gain (CG). For a metric score S,

$$\mathrm{CG} = S_{\text{corrected}} - S_{\text{uncorrected}}, \tag{7}$$

sign-adjusted so that positive always means the correction helped: unchanged for metrics where higher is better (balanced accuracy, AUPRC), and negated for loss-type metrics where lower is better (Brier score, log loss).

Correction Harm Rate (CHR). Across every stress cell tested for one method,

$$\mathrm{CHR} = \frac{\#\{\text{cells with CG} < 0\}}{\#\{\text{cells tested}\}}. \tag{8}$$

This counts any negative gain as harmful, including differences too small to distinguish from measurement noise. Section IV-A also reports a stricter variant that only counts a cell as harmful once CG falls below -0.01 on balanced accuracy, a sensitivity check on how much of the headline rate reflects negligible differences rather than a meaningful failure.

Regret. How large the harm is when a correction does fail, zero otherwise:

$$\mathrm{Regret} = \max(0,\ -\mathrm{CG}). \tag{9}$$

CHR answers how often a correction backfires. Regret answers how bad it is when it does, so a method with low CHR but large Regret on the cells where it fails carries a different practical risk than one with high CHR but small Regret.

Break-even contamination rate (c^*). For a method, dataset, and imbalance ratio, c^* is the smallest tested $c > 0$ at which the seed-averaged CG first drops to zero or below. This marks the first noise level a correction stops paying off, not necessarily the last, since CG is not guaranteed to stay negative at every higher c afterward (Section V-C quantifies how often it does not). If CG never crosses zero within the tested range, c^* is reported as greater than the largest tested c (0.30) rather than extrapolated beyond what was actually measured.

Prior Sensitivity Gap (PSG). For the three prior-dependent TabPFN methods,

$$\mathrm{PSG} = S_{\text{oracle prior}} - S_{\text{observed prior}}, \tag{10}$$

comparing the real, deployable variant against the oracle diagnostic described in Section III-E. A large positive PSG means most of a correction's failure traces specifically to the prior being wrong. A small PSG despite otherwise poor performance means the failure comes from elsewhere.

As a robustness check on whether pooling the balanced condition r=0.50 together with the three genuinely imbalanced ratios biases these headline contribution metrics, Section IV-C recomputes CHR, mean CG, break-even share, PSG, and every statistical test in Section III-H three times: once on the full pooled set (as everywhere else in this paper), once restricted to $r \in \{0.20, 0.10, 0.05\}$, and once restricted to r=0.50 alone. All three use the identical formulas defined above, applied to the same already-collected evaluations; no additional model fitting or scoring was performed.

H. Statistical Testing

For each model family (XGBoost, TabPFN) and each corruption direction, the methods in that family are compared using the Friedman test [53], a non-parametric omnibus test for differences among more than two related samples, applied to seed-averaged balanced accuracy across every (dataset, r, c) block with a complete row for every method in that family. For the three prior-dependent TabPFN methods, only the observed-prior variant, the same deployable score reported everywhere else in this study, enters the comparison. The oracle-prior rows are excluded so that a method's column reflects its deployable score alone, not an average of its observed and oracle variants. Blocks with fewer than ten complete rows are excluded as too few for a meaningful omnibus test. When the Friedman test is significant at $p < 0.05$, every pairwise comparison of methods within that family and direction is tested with the Wilcoxon signed-rank test [54], and the resulting p-values are corrected for multiple comparisons with Holm's step-down procedure [55].

Blocking by (dataset, r, c) treats every tested ratio and noise level within one dataset as an independent unit of evidence, but conditions drawn from the same dataset share its underlying feature distribution and test set, so they are not fully independent of each other. As a more conservative check, the same comparison is repeated with each dataset as a single block (n=26), first averaging balanced accuracy within a family and direction across every tested r and c for that dataset, then applying the Friedman test to those 26 dataset-level values per method, the standard practice for comparing algorithms across multiple datasets [56]. Every pairwise Wilcoxon comparison is repeated at this same dataset-level block, Holm-corrected separately from the (dataset, r, c) version, since a significant omnibus result does not

TABLE II
Software environment and frozen XGBoost configuration.

Component	Value
Software versions	
Python	3.13.5
Platform	macOS 26.5.1 (arm64)
NumPy	2.5.2
pandas	3.0.5
scikit-learn	1.9.0
SciPy	1.18.0
XGBoost	3.4.1
imbalanced-learn	0.14.2
statsmodels	0.14.6
TabPFN	8.3.0
Seeds	13, 37, 73, 5, 11, 17, 19, 23, 29, 31
XGBoost configuration (frozen, all variants)	
n_estimators	500
max_depth	6
learning_rate	0.05
subsample	0.8
colsample_bytree	0.8
eval_metric	logloss

by itself establish that every individual pairwise difference holds under the stricter standard. Section IV-I reports all versions.

I. Implementation and Reproducibility

Every stress cell (one dataset, seed, ratio, contamination rate, and direction) is repeated across ten seeds, $\{13, 37, 73, 5, 11, 17, 19, 23, 29, 31\}$, so no result depends on one particular random split or one particular set of flipped rows. A method failing on a given stress cell is logged with its error rather than silently skipped, and does not stop any other method or any later cell: one such failure occurred in the final run, tabpfn_downsample on APS Failure (seed 19, r=0.10, c=0.05, Direction A) raised a singular-value decomposition non-convergence error on that specific resampled training set, reproducibly across repeated attempts, and is reported as a failure rather than interpolated or dropped (Section IV). All other 112,319 of the 112,320 planned evaluations completed. Table II records the exact software versions, platform, and the frozen XGBoost configuration used throughout. The TabPFN checkpoint used was tabpfn-v3-classifier-v3_default.ckpt (SHA-256 d0d865d5…8f3988), and TabPFN inference used Apple's Metal (MPS) GPU backend rather than CPU. Every library's internal thread pool (OpenMP, BLAS) was forced to a single thread throughout the study, because XGBoost and PyTorch, which TabPFN depends on, each bundle their own threading runtime, and running both in the same process without this restriction reproducibly hangs (or, in the reverse import order, segfaults). At the dataset sizes used in this study, the resulting single-threaded slowdown is negligible.

IV. Results and Discussion

Across the 26 datasets, ten seeds, four imbalance ratios, and five contamination rates (the shared clean condition plus four noise levels in each of the two directions), each combination forms one of 9,360 stress cells. Nine methods (seven corrections plus the two uncorrected baselines) are scored on every cell, giving 84,240 evaluations. The three prior-dependent TabPFN corrections are additionally scored with the oracle prior for diagnostic purposes (Section III-E), adding 28,080 more, for 112,320 model-condition evaluations planned in total. 112,319 completed, one cell, tabpfn_downsample on APS Failure (seed 19, r=0.10, c=0.05, Direction A), failed to converge (Section III-I) and is excluded from the tables below rather than interpolated. All other results are computed over the full matrix.

TABLE III
Correction Harm Rate and mean Correction Gain (balanced accuracy) by method and direction, over noisy stress cells only ($c > 0$), pooled across all 26 datasets and ten seeds (n=416 per method/direction).

	CHR		mean CG	
Method	Dir. A	Dir. B	Dir. A	Dir. B
xgb_weighted	0.130	0.060	0.026	0.034
xgb_ros	0.147	0.067	0.026	0.035
xgb_smote	0.135	0.072	0.023	0.033
tabpfn_threshold	0.048	0.024	0.072	0.093
tabpfn_distpfn	0.351	0.438	0.009	−0.004
tabpfn_distpfn_t	0.339	0.474	0.008	−0.009
tabpfn_downsample	0.204	0.077	0.054	0.073

A. Overall Correction Robustness

Every one of the seven correction methods produced at least one stress cell with negative Correction Gain, confirming the pattern anticipated in Section I. No correction remains uniformly beneficial once label error is present, and how often this happens varies sharply by method. Table III summarizes Correction Harm Rate (CHR) and mean Correction Gain (CG, on balanced accuracy) for every method in each direction. tabpfn_threshold is the most robust method by both measures in both directions. Its CHR is 4.8% under Direction A and 2.4% under Direction B, less than half the next-lowest correction under either direction, and its mean CG (0.072 and 0.093) is the highest of any method in both directions. The worst end of the table looks different by direction, but in both directions it is the same two methods: tabpfn_distpfn and tabpfn_distpfn_t hold the highest CHR under Direction A (35.1% and 33.9%) and, by a wider margin, under Direction B (43.8% and 47.4%, more than five times the next-highest method there). Pooling across all 26 datasets separates the DistPFN pair clearly from the rest of the field in both directions, not just one.

Table III's CHR counts any negative gain as a failure, including differences too small to plausibly reflect anything but measurement noise. Requiring at least a 0.01 balanced-accuracy-point drop before counting a cell as harmful changes the magnitude sharply but not the ranking: tabpfn_threshold's harm rate falls to 0.2% (Dir. A) and

0.0% (Dir. B), every XGBoost method falls to 2.4% or below in both directions, and tabpfn_downsample falls to 6.5%/1.4%. The DistPFN pair are the one exception in both directions: 13.9%/13.2% under Direction A and 28.9%/28.9% under Direction B, four to over a hundred times any other method's rate at that threshold. Magnitude filtering shrinks the raw numbers substantially, but it does not change which two methods still fail meaningfully often.

Figure 2 makes the same contrast spatially, comparing tabpfn_threshold (top row) against tabpfn_distpfn (bottom row) across every tested combination of imbalance ratio r and noise level c. tabpfn_threshold's gain stays positive across the entire grid in both directions. tabpfn_distpfn's harm is not spread evenly. Under Direction B it is negative across nearly the entire r=0.50 row regardless of noise level, and only turns positive at lower ratios; under Direction A the picture is milder, staying pale-to-mildly-positive almost everywhere, with the clearest positive gain concentrated at high c and low r instead.

B. Simplicity Outperforms Sophistication

tabpfn_threshold does less than any other correction tested. It shifts the decision boundary from 0.5 to the class prior and nothing more (Section III-E). Despite that, it is the most robust method in the study on discrimination (Table III, Section IV-A). It has the lowest CHR and the highest mean CG in both directions, and its calibration is identical to its own baseline's by construction, since a threshold shift never touches the underlying predicted probabilities (Section IV-K returns to why that is not the same as leading on calibration outright). It leads on break-even share too (Table V, Section IV-F), so its advantage is not confined to CHR and mean CG alone. tabpfn_distpfn and tabpfn_distpfn_t sit at the opposite end (Table III again). Both rescale TabPFN's predictions against a prior and a temperature term, and hold the highest CHR of any method in both directions.

This finding builds on prior work rather than standing alone. Elor and Averbuch-Elor [12] found that on clean, uncorrupted data, a well-tuned classifier often gains nothing from balancing the training set, and can lose performance from it. This study's contribution is to show that the same pattern holds, and grows sharper, once label noise is added. A correction is not just sometimes unnecessary. It can actively backfire, and in this comparison the two most mathematically elaborate corrections, DistPFN and DistPFN-T, backfire hardest on discrimination, while the simplest correction backfires least. Much of that backfiring is concentrated at the balanced control ratio r=0.50, though not all of it, restricted to genuinely imbalanced training sets ($r\neq$0.50), the DistPFN pair's Direction B Correction Harm Rate falls from 43.8%/47.4% pooled to 29.8%/37.8%, still four to twelve times tabpfn_threshold's 0.3% there (Section IV-C). The ranking does not change at any scope. A practitioner choosing a correction method for a TabPFN deployment under uncertain label quality and optimizing for balanced accuracy has, based on this study's results, no clear reason to prefer a more elaborate posterior-rescaling method over a simple threshold shift. That recommendation does not extend to every deployment, however, a point Section IV-K returns to.

There may be a second, more structural reason threshold correction leads here, not just a general case for simplicity. This study's severe-imbalance conditions combine imbalance with a real train and test prior mismatch, since the test partition keeps each dataset's natural class ratio while training is deliberately pushed down to r (Section III-B). Threshold correction is nothing but a prior-shift adjustment, and DistPFN and DistPFN-T target the same mismatch through a more elaborate mechanism. All three are therefore being tested in exactly the deployment scenario their underlying idea is built for, which may explain part of why prior-adjusting corrections outperform resampling and cost-sensitive weighting generally, separate from how well each one specifically tolerates label contamination on top of that shift. This coupling is broad-based, seven datasets sit at or below the two most severe tested ratios (Section III-B), so this structural explanation applies to a substantial share of the study. It does not explain why the two DistPFN variants fail so much more often than plain threshold correction while addressing a closely related problem. That gap is a matter of contamination robustness specifically, not of whether prior adjustment is the right tool for this design.

C. Robustness Check: Isolating the Balanced Control Condition (r=0.50)

Section IV-A's headline numbers pool all four tested imbalance ratios, including r=0.50, a perfectly balanced training set before contamination. r=0.50 is not itself an imbalanced condition, and Figure 2 already shows tabpfn_distpfn's harm concentrating specifically there under Direction B. This subsection isolates it directly, every contribution metric and statistical test from Sections IV-A–IV-I is recomputed twice more, once restricted to $r \in \{0.20, 0.10, 0.05\}$ (the genuinely imbalanced subset, hereafter $r\neq$0.50) and once restricted to r=0.50 alone (the balanced control), using the identical formulas and the same already-collected evaluations. No model was retrained or re-scored.

Table IV reports Correction Harm Rate and mean Correction Gain on the $r\neq$0.50 subset, directly comparable to Table III. tabpfn_threshold is unaffected in its standing, if anything strengthened, its Direction B CHR falls from 2.4% to 0.3%, and its mean CG rises in both directions. The two DistPFN variants change substantially but not completely. Under Direction B, pooled CHR is 43.8%/47.4%; restricted to $r\neq$0.50 it falls to 29.8% (tabpfn_distpfn) and 37.8% (tabpfn_distpfn_t), while their mean CG rises from a negative $-0.004/-0.009$

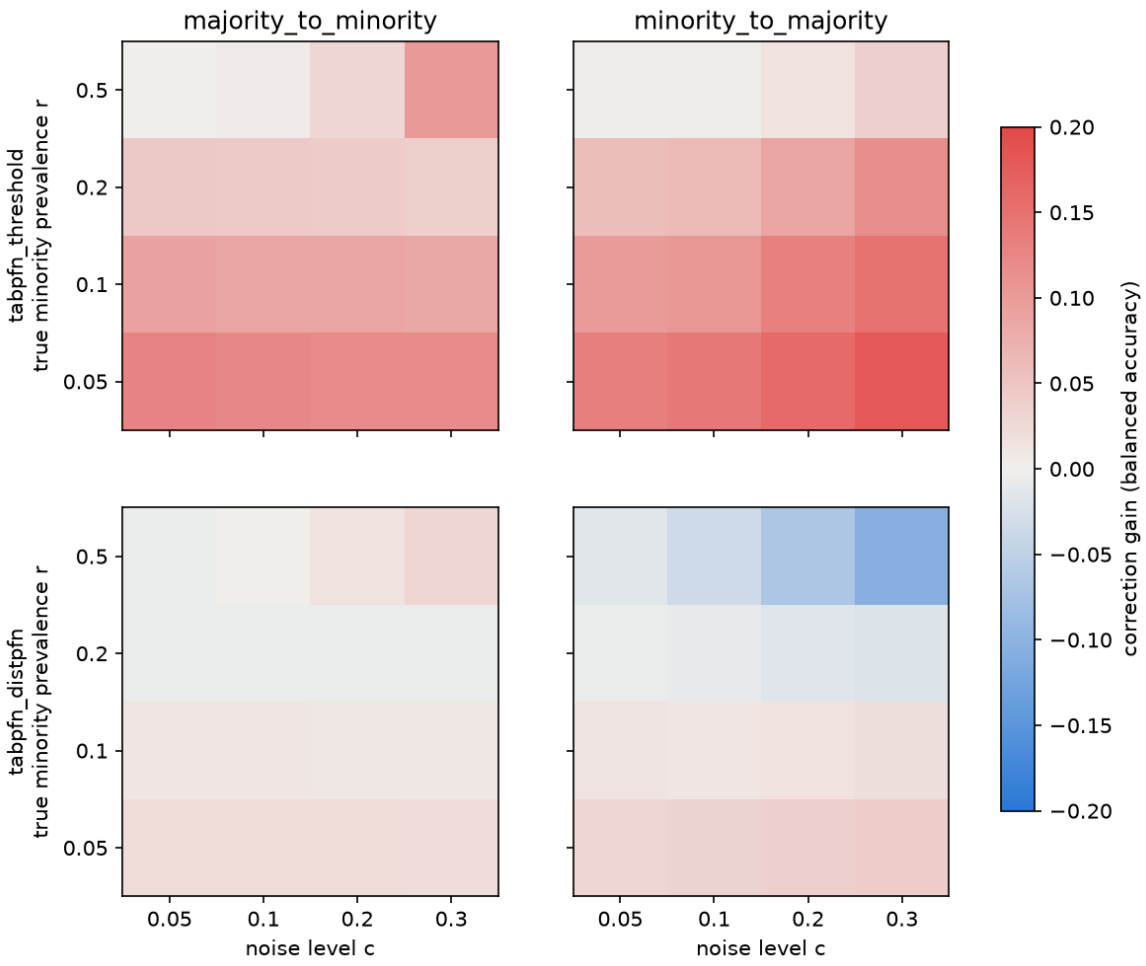


Fig. 2. Correction gain (balanced accuracy) across imbalance ratio r and noise level c, for the most robust corrected method (tabpfn_threshold, top) and the least robust (tabpfn_distpfn, bottom), pooled across all 26 datasets and ten seeds. Blue indicates the correction hurt relative to the uncorrected baseline at that cell, and red indicates it helped.

pooled to a positive 0.013/−0.004. Isolated to r=0.50 alone, the same two methods fail 85.6% and 76.0% of the time under Direction B, with a clearly negative mean gain (−0.055, −0.023), and rarely reach break-even at that ratio (3.8% of (dataset, r=0.50) pairs for both, versus 46.2%/47.4% at $r\neq$0.50). Under Direction A the shift is milder, CHR for the DistPFN pair moves from 35.1%/33.9% pooled to 34.9%/35.9% at $r\neq$0.50, essentially unchanged. tabpfn_downsample's CHR under Direction A does not fall when r=0.50 is excluded (20.4% pooled vs. 18.6% at $r\neq$0.50), consistent with Section IV-E's finding that its failures are not tied to any one imbalance ratio.

TABLE IV
Correction Harm Rate and mean Correction Gain (balanced accuracy), restricted to the genuinely imbalanced subset $r \in \{0.20, 0.10, 0.05\}$ (excludes the balanced control r=0.50; n=312 per method/direction). Directly comparable to Table III.

	CHR		mean CG	
Method	Dir. A	Dir. B	Dir. A	Dir. B
xgb_weighted	0.115	0.032	0.028	0.043
xgb_ros	0.115	0.026	0.029	0.044
xgb_smote	0.106	0.029	0.026	0.041
tabpfn_threshold	0.032	0.003	0.084	0.119
tabpfn_distpfn	0.349	0.298	0.009	0.013
tabpfn_distpfn_t	0.359	0.378	0.004	−0.004
tabpfn_downsample	0.186	0.051	0.061	0.094

Break-even and Prior Sensitivity results show a related but not identical pattern. At r=0.50, mean PSG for the DistPFN pair under Direction A falls to −0.058/−0.059, substantially more negative than the pooled figures in Section IV-G; restricted to $r\neq$0.50, PSG for both instead turns slightly positive under Direction A (+0.010, +0.009), meaning the oracle prior modestly helps once the balanced control is excluded, though it remains negative for both methods under Direction B (−0.013, −0.009) and for tabpfn_threshold in both directions regardless of r. Every Friedman omnibus test remains significant on both the $r\neq$0.50 and r=0.50-only subsets, at both blocking levels ($p < 10^{-8}$ throughout, Table VII–VIII report the pooled

version; the full per-view statistics accompany this paper's other results artifacts).

Taken together, the core claims survive this check intact, and one gets stronger, no correction is uniformly safe, method identity significantly affects balanced accuracy in every family and direction regardless of r=0.50's inclusion, and tabpfn_threshold remains the most robust method throughout, more clearly so once the balanced control is removed. What changes is the magnitude attributed to the DistPFN pair's failure under Direction B specifically, a real share of it, but not the majority, is a property of the balanced condition rather than of imbalance severity, since the pair's Direction B CHR only falls by roughly a third (43.8%→29.8% and 47.4%→37.8%) once r=0.50 is excluded, not to the low single digits.

D. Direction-Dependent Failure

Failure rates differ substantially between the two directed corruption mechanisms, and the direction of that difference is itself method-dependent rather than uniform. Five of the seven methods have a higher CHR under Direction A (majority rows relabeled as minority) than under Direction B (minority rows relabeled as majority). These are xgb_ros, xgb_smote, xgb_weighted, tabpfn_downsample, and tabpfn_threshold, the last two by a wide margin (Table III). The two DistPFN variants show the opposite pattern, failing more often under Direction B than Direction A. Figure 3 shows CHR and mean regret side by side for every method, split by direction.

Regret and CHR tell a consistent story: the DistPFN pair carries both the highest CHR and the highest mean regret in both directions. Under Direction A, tabpfn_distpfn_t's mean regret (0.0108) is the highest of any method, ahead of tabpfn_distpfn (0.0059) and tabpfn_downsample (0.0023), even though tabpfn_downsample's CHR (20.4%) sits between the DistPFN pair's (33.9–35.1%) and the XGBoost methods' (13.0–14.7%). Under Direction B the gap widens sharply, tabpfn_distpfn_t's regret (0.0292) and tabpfn_distpfn's (0.0239) are an order of magnitude above every other method, all of which sit at 0.0004 or below. tabpfn_threshold has the lowest regret of any method in both directions (0.0001 and 0.0000), consistent with its lead on every other measure in this study.

Part of this asymmetry traces back to the uncorrected baselines themselves. Comparing balanced accuracy at c=0 against c=0.30 for xgb_none, Direction B degrades performance by 0.076 (0.741 to 0.665) versus 0.034 for Direction A (0.741 to 0.707), roughly 2.2 times as much. tabpfn_none shows the same asymmetry, 0.066 (0.765 to 0.699) versus 0.032 (0.765 to 0.733), roughly 2.0 times as much. Losing genuine minority rows (Direction B) is inherently more damaging to an uncorrected model than diluting the minority class with mislabeled majority rows (Direction A), which helps explain why a correction's apparent robustness under one direction does not predict its robustness under the other.

E. Direction Asymmetry Undercuts a Single Robustness Score

Five of seven methods fail more often under Direction A (majority rows relabeled as minority) than Direction B (minority rows relabeled as majority), while the two DistPFN variants show the opposite pattern (Table III, Section IV-D). This asymmetry is exactly what the label-noise literature's class-conditional and asymmetric-noise formalizations predict in principle [13], [14], but this study's results show it is not a minor theoretical caveat. A method's CHR under one direction is a poor predictor of its CHR under the other. tabpfn_downsample's CHR is 20.4% under Direction A and 7.7% under Direction B (Table III), nearly a threefold difference in the same method on the same datasets. A robustness claim validated under only one corruption mechanism, or under the symmetric random-flip noise more commonly studied, does not transfer to a directed, class-conditional error of the kind this study injects, and in practice the true direction of label error is rarely known in advance. This is the strongest argument in this study for testing every correction under both directions rather than one.

F. Break-Even Contamination Rates

Table V reports, for each method and direction, the share of the 104 (dataset, r) combinations (26 datasets $\times$ 4 ratios) whose seed-averaged Correction Gain never dropped to zero or below within the tested range, i.e., $c^* > 0.30$. tabpfn_threshold leads here too, staying net-beneficial through the full range in 85.6% of Direction A conditions and 87.5% of Direction B conditions, the highest share of any method in both directions, extending its lead from CHR and mean CG to this measure as well. tabpfn_distpfn and tabpfn_distpfn_t sit at the bottom in both directions, the lowest share in both directions belongs to the DistPFN pair, 40.4% (tabpfn_distpfn) under Direction A and 32.7% (tabpfn_distpfn_t) under Direction B. CHR counts every failing cell individually, while break-even share only asks whether a method's seed-averaged trend, aggregated within one (dataset, r) combination, ever crosses zero, a coarser question that mostly, but not perfectly, agrees with CHR's answer. That distinction is exactly why Section III-G defines CHR, Regret, and c^* as separate, complementary measures rather than collapsing robustness into one number.

G. Prior Sensitivity

The corrupted prior explains less of a correction's failure than expected. On average, the deployable, observed-prior variant slightly outperforms the oracle diagnostic rather than trailing it. Table VI reports mean PSG (Equation 10), both pooled across all noisy cells and restricted to the highest tested contamination, c=0.30,

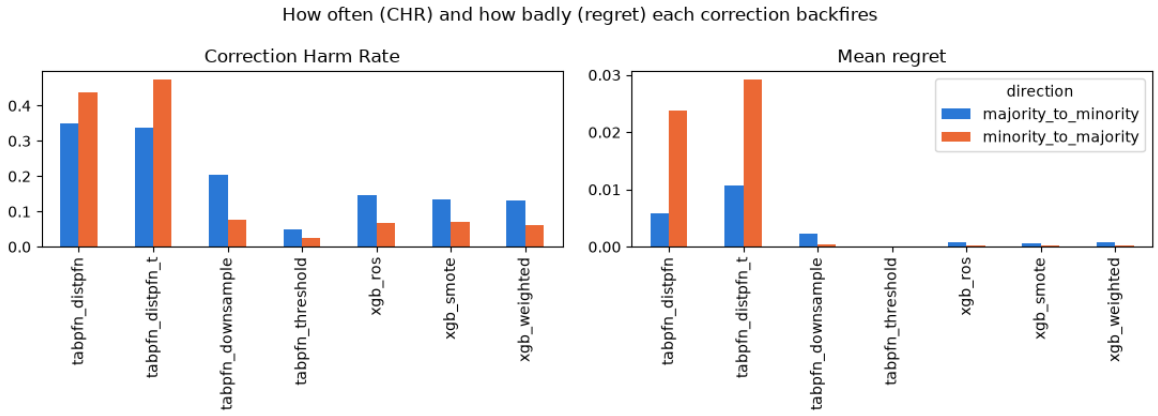


Fig. 3. Correction Harm Rate (left) and mean regret (right, $\max(0, -\mathrm{CG})$) by method and direction, over noisy stress cells only, pooled across all 26 datasets. tabpfn_distpfn and tabpfn_distpfn_t have both the highest CHR and the highest mean regret under both directions, most severely under Direction B. tabpfn_threshold has the lowest of both in both directions.

TABLE V
Share of (dataset, r) conditions, out of 104 (26 datasets × 4 ratios), where a method's seed-averaged Correction Gain never dropped to zero or below within the tested contamination range ($c^* > 0.30$).

Method	Dir. A	Dir. B
xgb_weighted	69.2%	80.8%
xgb_ros	64.4%	80.8%
xgb_smote	62.5%	78.8%
tabpfn_threshold	85.6%	87.5%
tabpfn_distpfn	40.4%	37.5%
tabpfn_distpfn_t	41.3%	32.7%
tabpfn_downsample	62.5%	74.0%

TABLE VI
Mean Prior Sensitivity Gap (Equation 10) by method and direction, pooled across all noisy cells and restricted to the highest tested contamination, c=0.30, alone. Negative throughout: the oracle prior does not reliably outperform the observed one.

Method / Direction	mean PSG (pooled)	mean PSG (c=0.30)
tabpfn_threshold, Dir. A	−0.012	−0.037
tabpfn_threshold, Dir. B	−0.008	−0.019
tabpfn_distpfn, Dir. A	−0.007	−0.033
tabpfn_distpfn, Dir. B	−0.020	−0.043
tabpfn_distpfn_t, Dir. A	−0.008	−0.036
tabpfn_distpfn_t, Dir. B	−0.014	−0.033

alone. Pooled, it is negative for all three prior-dependent methods in both directions. Under Direction A it is −0.012 for tabpfn_threshold, −0.007 for tabpfn_distpfn, and −0.008 for tabpfn_distpfn_t, with tabpfn_threshold the largest of the three in magnitude despite being the most robust method on balanced accuracy. Under Direction B the gap is −0.008, −0.020, and −0.014 respectively. This gap is not fixed, however, and the ranking across methods it implies is not stable either. Restricted to c=0.30 only, mean PSG for tabpfn_threshold widens to −0.037 under Direction A, now the widest of the three at that noise level, ahead of tabpfn_distpfn_t (−0.036) and tabpfn_distpfn (−0.033), a reversal of which method has the largest prior-sensitivity gap once contamination is severe compared to the pooled figures above.

Figure 4 shows the observed and oracle curves for all three methods. At low contamination they stay close together, but for the two DistPFN variants they separate sharply above c=0.20 under Direction A, with the oracle variant degrading faster than the observed one. tabpfn_threshold shows a comparable or larger separation at the highest tested noise level specifically, despite a much smaller gap at lower contamination. Even using the true, uncorrected prior does not fully recover a correction's clean-condition performance once contamination is severe. This means the corrupted prior is not the main reason these corrections decline at high contamination, for any of the three methods.

H. The Corrupted Prior Is Not the Main Failure Mode

The most surprising result of this study is that mean PSG is negative for all three prior-dependent TabPFN methods in both directions (Section IV-G). The true, uncorrupted prior, something no real deployment could ever supply, does not reliably outperform the corrupted, observed one. Under Direction A, at high contamination, the oracle variant of all three prior-dependent methods degrades by a comparable or larger amount than the deployable one (Figure 4). If the wrong prior were the primary cause of these methods' decline, correcting it should have helped. It does not, at least not under Direction B or, at high contamination, under Direction A either. Restricting to the genuinely imbalanced subset ($r \neq 0.50$) does flip mean PSG positive for both DistPFN variants under Direction A (+0.010, +0.009), meaning the corrupted prior does contribute something to their Direction A decline once the balanced control is excluded, it is simply not the dominant effect the pooled, more strongly negative figure (−0.058/−0.059 at r=0.50 alone) would

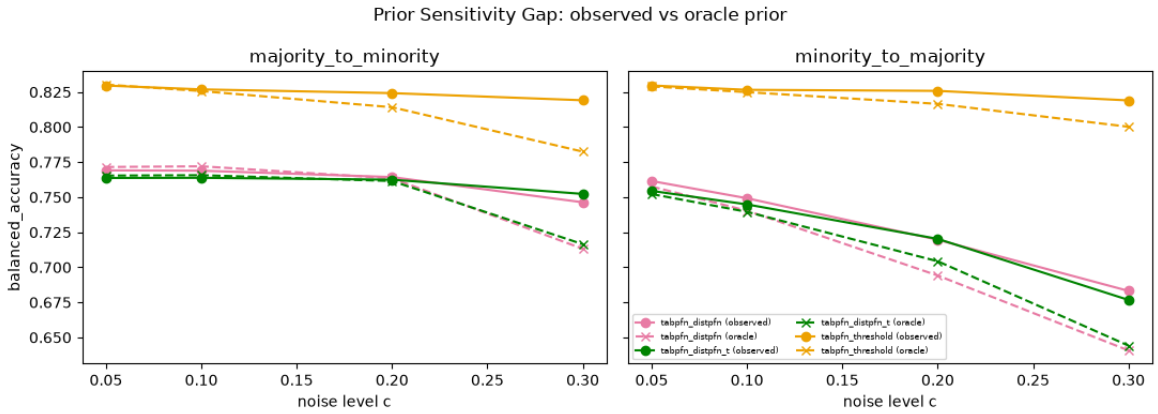


Fig. 4. Observed-prior (deployable, solid) versus oracle-prior (diagnostic, dashed) balanced accuracy for the three prior-dependent TabPFN methods, pooled across all 26 datasets. The oracle variant is never reliably better than the observed one, and all three methods show comparable degradation between the two at the highest tested contamination.

suggest (Section IV-C). The claim that the corrupted prior is not the primary failure mode is therefore strongest for Direction B and, increasingly at high contamination, for tabpfn_threshold too, and more qualified for the DistPFN pair under Direction A at moderate contamination.

One structural reason follows directly from how these corrections work. tabpfn_threshold, tabpfn_distpfn, and tabpfn_distpfn_t all adjust TabPFN's output probabilities after the fact, using the class prior [10], [3]. None of them touch the training rows TabPFN actually reads as its in-context prompt [1], [2]. Contamination happens before any correction is applied, so the mislabeled rows stay in that prompt regardless of which prior a posterior-rescaling method is given afterward. Swapping in the true prior fixes the summary statistic the correction uses, but not the individual wrong examples TabPFN has already been shown. This would explain why the oracle prior helps so little, though confirming it directly would require an experiment this study did not run, comparing a prior-only fix against a fix that also removes the mislabeled rows from the context itself.

TABLE VII
Friedman omnibus test statistics by model family and corruption direction, blocked by (dataset, r, c), n=416. All four are significant at $p < 10^{-85}$.

Family	Direction	χ^2	p
XGBoost	Dir. A	400.0	2.2×10^{-86}
XGBoost	Dir. B	563.7	7.3×10^{-122}
TabPFN	Dir. A	681.7	3.1×10^{-146}
TabPFN	Dir. B	933.0	1.2×10^{-200}

TABLE VIII
Friedman omnibus test statistics using dataset-level blocks (n=26, one per dataset, mean balanced accuracy pooled across every tested ratio and noise level within that dataset), a more conservative robustness check on Table VII.

Family	Direction	χ^2	p
XGBoost	Dir. A	46.8	3.7×10^{-10}
XGBoost	Dir. B	47.4	2.9×10^{-10}
TabPFN	Dir. A	69.4	3.0×10^{-14}
TabPFN	Dir. B	71.5	1.1×10^{-14}

I. Statistical Significance

Table VII reports the Friedman omnibus test for each model family and corruption direction, blocked by (dataset, r, c) (n=416). All four are significant at $p < 10^{-85}$, so method identity significantly affects balanced accuracy within both families in both directions.

Blocking by (dataset, r, c) treats conditions drawn from the same dataset as independent evidence, though they share that dataset's underlying feature distribution and test set. Table VIII repeats the same comparison with each dataset as a single block (n=26) instead, the more conservative standard for comparing algorithms across multiple datasets (Section III-H). The p-values increase by many orders of magnitude under this stricter treatment, from below 10^{-85} to between 10^{-14} and 10^{-9}, but all four tests remain significant far below the conventional $p < 0.05$ threshold. Method identity affects balanced accuracy either way. The (dataset, r, c) version overstates how strong that evidence is, not whether it exists.

At the (dataset, r, c) block level, every uncorrected baseline differs significantly from every correction in its family under Direction A (Holm-corrected). Under Direction B, this holds for every XGBoost correction against xgb_none and for tabpfn_threshold and tabpfn_downsample against tabpfn_none, but not for tabpfn_distpfn or tabpfn_distpfn_t against tabpfn_none ($p_{\text{Holm}} = 0.58$ and 0.87): at this block level, on the full 26-dataset pool, neither DistPFN variant is statistically distinguishable from doing nothing at all under Direction B, despite Table III's pooled CHR of 43.8%/47.4% for the same comparison. Seven of the 32 pairwise comparisons are non-significant overall at this block level, the two

just named, plus two pairs among the three XGBoost corrections under Direction A (xgb_weighted vs. xgb_ros, xgb_ros vs. xgb_smote) and all three XGBoost pairs under Direction B.

Repeating every pairwise comparison with each dataset as a single block (n=26) sharpens this pattern rather than reversing it. Only 21 of the 32 comparisons remain significant. tabpfn_threshold still differs significantly from every other TabPFN method in both directions, so the finding that it is the most robust method survives the stricter standard intact. The three XGBoost corrections become mutually indistinguishable from each other in both directions at this block level (all three pairwise comparisons non-significant each way), though each still differs significantly from xgb_none. Among the TabPFN family, tabpfn_distpfn is not significantly different from tabpfn_none under Direction A either at this stricter level ($p_{\text{Holm}} = 0.07$), and tabpfn_distpfn versus tabpfn_distpfn_t also loses significance under Direction A ($p_{\text{Holm}} = 0.12$). Under Direction B, the collapse is complete: tabpfn_none, tabpfn_distpfn, and tabpfn_distpfn_t are mutually indistinguishable from each other at this block level ($p_{\text{Holm}} = 1.00$ for all three pairwise comparisons), meaning the stricter, dataset-level test finds no statistical evidence that either DistPFN variant differs from doing nothing at all, or from each other, under Direction B. None of the descriptive measures behind these comparisons, CHR, mean CG, or break-even share, depend on this test. What changes is which pairwise differences a formal significance test can support at n=26, not the underlying data those tests were run on, but the change is large enough here to matter for how the DistPFN pair's Direction B failure should be described, real and large by the descriptive measures, but not statistically distinguishable from the null at the more conservative dataset-level standard.

J. Calibration Reliability

Figure 5 tracks Brier score and expected calibration error against noise level for every method. Every XGBoost correction improves on xgb_none's Brier score and ECE at every noise level tested, in both directions, with no exception. The TabPFN family shows a different kind of split. tabpfn_downsample is worse-calibrated than the uncorrected tabpfn_none baseline on both ECE and Brier score at every noise level tested under Direction A, the only method in the study with that pattern on either metric. tabpfn_threshold never exceeds its baseline's Brier score or ECE in either direction, which follows directly from how it works, it only moves the decision threshold, so the underlying predicted probabilities that Brier score and ECE measure are identical to tabpfn_none's by construction. tabpfn_distpfn and tabpfn_distpfn_t do not merely avoid backfiring on calibration, they are the two best-calibrated methods of any tested, correction or baseline, at every noise level in both directions, with no exception. This holds despite the same two methods having the highest Correction Harm Rate in the study on balanced accuracy (Section IV-A): a DistPFN correction that more frequently underperforms its own uncorrected baseline on balanced accuracy can still be assigning its predictions a well-calibrated probability, discrimination and calibration are not the same failure mode. tabpfn_threshold remains consistent with its standing as the most robust method on balanced accuracy (Sections IV-A and IV-D), but on raw calibration it trails the two DistPFN variants throughout.

K. Accuracy and Calibration Answer Different Questions

Every XGBoost correction improves calibration over its baseline at every noise level tested, even the ones that sometimes hurt balanced accuracy (Figure 5, Section IV-J). Under Direction A, tabpfn_downsample is worse-calibrated than doing nothing on both expected calibration error and Brier score at every noise level tested. tabpfn_distpfn and tabpfn_distpfn_t take the opposite path entirely. The same two methods with the highest Correction Harm Rate in the whole study (Table III) are also its two best-calibrated methods, correction or baseline, at every noise level in both directions. A method that more often underperforms its own uncorrected baseline on discrimination can still be assigning its predictions a well-calibrated probability. Balanced accuracy and calibration measure different properties of a classifier, and this study's results show a correction can excel at one while failing badly at the other, not merely land somewhere middling on both. This matters for deployment. A system that only needs a correct label, such as most classification benchmarks, cares primarily about balanced accuracy or a related discrimination metric, and by that standard DistPFN and DistPFN-T are a poor choice under contamination. A system that acts on a predicted probability (setting a risk threshold, allocating a limited budget, ranking cases by urgency) cares about calibration first, and for that use case their poor CHR would be the wrong reason to rule them out. Reporting CHR and mean CG alone for the headline comparisons would also miss that tabpfn_downsample is uniquely bad at calibration under Direction A even though its balanced accuracy profile looks far more favorable there, the second-highest mean CG of any method. Evaluating a correction method for deployment requires checking both axes separately, not inferring one from the other, and not assuming a method's rank on one predicts its rank on the other.

L. Implications for Imbalance Correction Under Label Noise

Related work on class imbalance and label noise as a joint problem has so far proposed new training-time algorithms, combining cleaning with resampling [18], active learning with noisy-label training [21], class-balanced sample selection [22], or, inverting the relationship entirely, deliberately engineered label noise designed to fix a known

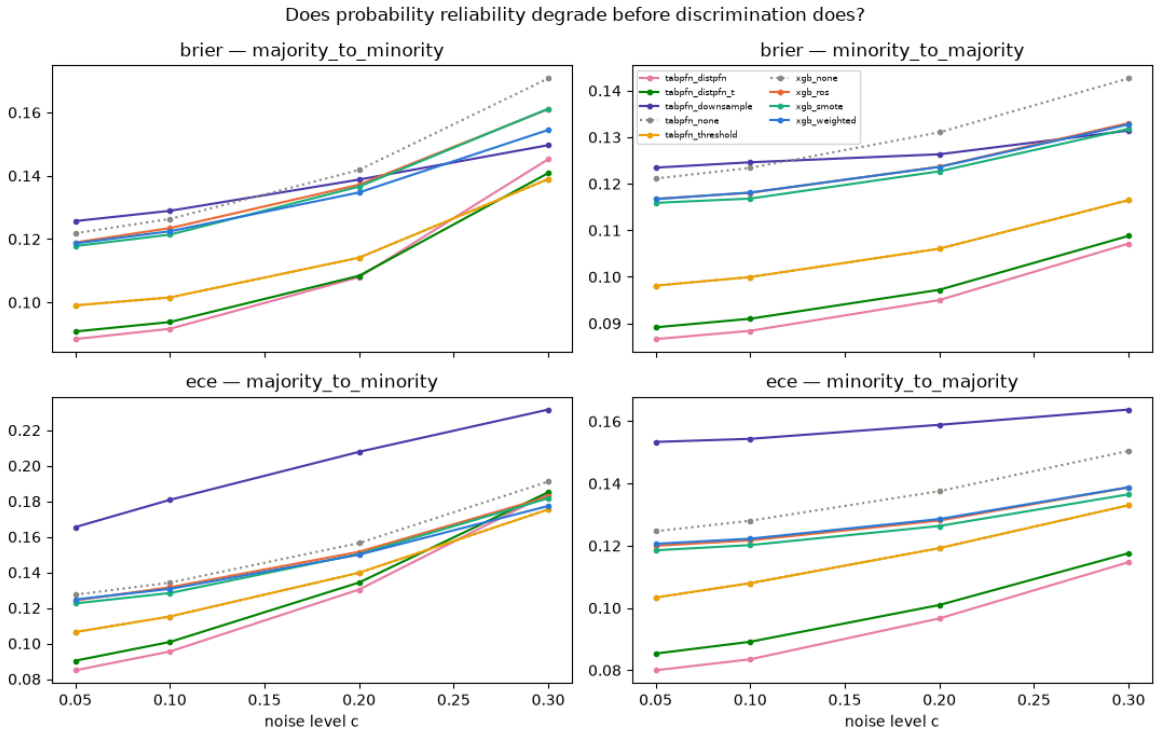


Fig. 5. Brier score (top) and expected calibration error (bottom) against noise level, faceted by direction, pooled across all 26 datasets. Dotted gray lines are the two uncorrected baselines. Every XGBoost correction improves on its baseline at every noise level. Within the TabPFN family, tabpfn_downsample is the only correction that crosses above its own baseline, under Direction A, while tabpfn_distpfn and tabpfn_distpfn_t stay the best-calibrated methods in the figure throughout.

class bias [23]. This study starts from a different question. It asks whether the corrections practitioners are already using need replacing at all. The answer depends on which correction is already in use. tabpfn_threshold stays net-beneficial through the full contamination range in 85.6% to 87.5% of the conditions tested (Table V), the highest share of any method in both directions, so switching to it may fix most of what a new algorithm would otherwise need to solve. tabpfn_distpfn and tabpfn_distpfn_t fail badly enough on discrimination under high contamination, in both directions on this study's full dataset pool, that the case for joint noise-and-imbalance-aware training remains strong, for a deployment that needs a correct label. A deployment that instead needs a well-calibrated probability is a different case entirely (Section IV-K), and swapping either DistPFN variant out for a different correction there could trade away its best property to fix its worst one. Telling these situations apart requires the kind of audit this study performs, not an assumption that a published correction is safe by default, or unsafe by default, on every axis a deployment might care about.

V. Threats to Validity

A. Internal Validity

Every tabpfn_* result in this paper comes from one checkpoint, verified by SHA-256 hash (Section III-I), and uses Apple's Metal GPU backend for inference. The whole study, XGBoost included, ran on one machine with every library forced to a single thread to avoid a documented XGBoost/PyTorch conflict. A different checkpoint, backend, or thread count could shift specific numbers, but there is no reason to expect the broader pattern, that robustness varies by method and direction, to depend on any of them. Each stress cell was repeated across ten seeds to reduce dependence on a single train/test split or corruption realization. Ten repetitions still provide only limited precision for quantities determined by whether a seed-averaged value crosses zero, including Correction Harm Rate and the break-even rate. Additional seeds would further improve uncertainty characterization of both measures, a limitation Section V-C's break-even non-monotonicity finding may partly reflect.

The claim that robustness varies by direction carries its own caveat. Direction A's and Direction B's contamination rate c are defined against different reference points (Section III-D), so a matched nominal c does not correspond to a matched absolute flip count between the two. Direction A flips a larger fraction of $n_{\min}$'s worth of rows than Direction B at every tested c above zero. The finding that direction changes how often a correction fails should be read as a comparison between two distinct, realistically motivated corruption processes at their own natural severity levels, not as a magnitude-controlled ma-

nipulation of direction alone. A shared flip-count definition was considered and rejected, since the two most natural candidates either require flipping more minority rows than exist at r=0.05, or introduce an r-dependent mismatch between directions that grows increasingly severe as r decreases. That mismatch would be worst precisely at the most imbalanced settings this study emphasizes, worse than the fixed, r-independent gap the current design carries at every tested r.

A related caveat applies to the imbalance ratio r itself. The test partition is never resampled and keeps each dataset's natural class distribution (Section III-B), while training is deliberately pushed to r, so the low-r conditions this study emphasizes combine imbalance with train and test prior shift rather than isolating imbalance alone. This is the standard evaluation paradigm across the class-imbalance literature. Training on a re-balanced set and testing on the natural population is what makes a correction's benefit deployment-relevant in the first place, but it means this study cannot cleanly attribute a method's robustness to tolerance of imbalance specifically as opposed to tolerance of prior shift. This matters most for tabpfn_threshold, tabpfn_distpfn, and tabpfn_distpfn_t, whose underlying mechanism directly targets prior mismatch (Section IV-B). A design that resampled the test partition to match each tested r would remove this coupling, at the cost of evaluating performance on a population that does not correspond to any dataset's actual distribution. Section IV-C partially addresses a related but distinct concern, whether the balanced ratio r=0.50 (not imbalanced even before contamination) was itself driving the pooled headline numbers, by recomputing every contribution metric and statistical test with r=0.50 isolated from $r \in \{0.20, 0.10, 0.05\}$. The results confirm that a substantial share of the DistPFN pair's apparent harm under Direction B is concentrated at that one ratio. This does not resolve the train/test prior-shift coupling described above, which remains present within the $r \neq 0.50$ subset itself, but it does rule out r=0.50 specifically as an unexamined confound in the headline comparisons.

B. External Validity

The 26 datasets audited range from 270 to 60,000 raw rows (Table I), but the raw row count is not what any method actually fits on. The controlled training set every stress cell draws from is capped by N_{common} (Section III-C), which tops out at 18,698 rows even for Adult, the dataset with the largest minority class in raw terms, because minority availability at r=0.50 is the binding constraint regardless of how many majority rows the raw dataset has. Every condition tested therefore stays well within TabPFN's intended small-tabular-data regime but far from the scale where gradient-boosted methods are typically deployed. Whether the same robustness ranking holds on larger, higher-dimensional, or more categorical-heavy tabular data, or simply on a dataset whose minority class alone is large enough to push N_{common} higher, is untested. Every dataset audited is also a binary-classification problem. Whether the same corrections and the same robustness patterns extend to multi-class imbalance, where a correction may need to reweight several minority classes at once rather than one, is untested as well.

The two corruption mechanisms audited are both class-conditional (Section III-D). The flip probability depends only on the true class, not on the specific feature values of the row being flipped. Real-world label noise is often instance-dependent instead, arising from cases that are genuinely ambiguous or that share features with the other class, and this study says nothing about whether the same methods stay robust under that kind of noise. The four tested contamination rates also stop at 0.30, with results never extrapolated beyond that point (Section III-G), so this study cannot say whether the same patterns hold at higher contamination. Each stress cell also applies exactly one corruption direction at a time, never a mixture of both within the same corrupted training set. Real-world label noise plausibly combines both directions at once, and this study does not test whether a correction's robustness under one direction in isolation predicts its robustness once both are present together.

C. Construct Validity

Correction Gain, Correction Harm Rate, Regret, and the break-even rate are original to this paper. They are reported here only on balanced accuracy, not on the other nine metrics tracked in Section III-F. A correction ranked robust by CHR on balanced accuracy is not guaranteed to be equally robust by the same measure on AUPRC or minority recall, a point the calibration results in Section IV-J already illustrate for a different metric pair. The break-even rate itself assumes a correction's gain declines monotonically as contamination rises, an assumption this study's own data does not fully support. Of the 522 (method, dataset, ratio, direction) combinations with a defined break-even point, 54.6% show the seed-averaged gain recover to positive at a higher contamination level after already crossing zero at a lower one, a majority, not a minority, of defined break-even points. c^* marks the first noise level a correction stops paying off, not a guarantee that it stays that way at every higher level tested. The Prior Sensitivity Gap's oracle variant assumes access to the true, pre-corruption class prior, something no real deployment has. It is a diagnostic construct, not a claim about achievable performance, and Section IV-G found it is not even reliably better than the observed-prior variant it is meant to bound.

VI. Conclusion and Future Work

This paper set out to test whether standard imbalance corrections remain worth applying once the labels they depend on cannot be trusted. Across 26 datasets, seven

correction methods spanning two model families, and two directed label-error mechanisms, every correction audited underperformed its matched uncorrected baseline in at least some tested condition. How often and how badly it did so depended on the method, on the direction of the label error, and on whether balanced accuracy or calibration was the metric being judged, never on a single number alone. The simplest correction tested, a threshold shift keyed to the class prior, was consistently the most robust on discrimination, leading every other method on Correction Harm Rate and mean Correction Gain in both directions. The two most mathematically involved corrections, DistPFN and DistPFN-T, held the highest Correction Harm Rate of any method under both directions, more than five times the next-highest method's rate under Direction B, the opposite of what their added sophistication might suggest, yet those same two corrections were also the best-calibrated methods in the entire study. A robustness check restricted to genuinely imbalanced training sets, excluding the balanced r=0.50 control, shows a real share, though not the majority, of that harm rate concentrated at the balanced condition specifically: the DistPFN pair's Direction B Correction Harm Rate falls from 43.8%/47.4% to 29.8%/37.8% once r=0.50 is excluded, while tabpfn_threshold's lead over the rest of the field holds or strengthens throughout. A correction that more frequently degrades discrimination relative to its uncorrected baseline can nevertheless produce better-calibrated probabilities, a split this paper's results surface rather than resolve in either method's favor.

The five contribution metrics introduced here, Correction Gain, Correction Harm Rate, Regret, the break-even contamination rate, and the Prior Sensitivity Gap, made this possible, separating whether a correction helps from how often it fails, how badly it fails when it does, and how much of that failure traces to the corrupted prior specifically. The same framework also surfaced a result this paper did not anticipate, that the corrupted prior is not the primary reason TabPFN's posterior-rescaling corrections decline under contamination, since handing them the true prior did not reliably help, and on average even hurt slightly. Section IV-H offers one explanation, that these corrections never touch the mislabeled rows still sitting in TabPFN's in-context prompt.

Several directions follow from what this study leaves open. Confirming the explanation above would need an experiment that cleans the mislabeled rows from TabPFN's in-context prompt directly, rather than only adjusting the prior applied to it; separately, because training here is deliberately pushed to each tested ratio while the test partition keeps each dataset's natural class distribution, a second evaluation regime that resamples the test partition to match would isolate how much of a prior-adjusting method's robustness, particularly tabpfn_threshold's, reflects imbalance tolerance specifically rather than the train/test prior mismatch this design creates, a regime already designed and implemented at the code level but not yet run at full scale given its compute cost. Beyond those two, testing whether the same robustness ranking holds on a dataset whose minority class alone permits a larger controlled training set, on tabular foundation models other than TabPFN, under multi-class imbalance, and under instance-dependent or simultaneous-direction label noise rather than the class-conditional, single-direction mechanisms audited here, are the most direct ways to extend this audit.

References

[1] N. Hollmann, S. Müller, K. Eggensperger, and F. Hutter, "TabPFN: A transformer that solves small tabular classification problems in a second," in International Conference on Learning Representations (ICLR), 2023.

[2] N. Hollmann, S. Müller, L. Purucker, A. Krishnakumar, M. Körfer, S. B. Hoo, R. T. Schirrmeister, and F. Hutter, "Accurate predictions on small data with a tabular foundation model," Nature, vol. 637, no. 8045, pp. 319–326, 2025.

[3] S. Lee, "Mitigating label shift in tabular in-context learning via test-time posterior adjustment," in International Conference on Machine Learning (ICML), 2026, arXiv:2605.04363.

[4] T. Chen and C. Guestrin, "XGBoost: A scalable tree boosting system," in Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining (KDD), 2016, pp. 785–794.

[5] N. V. Chawla, K. W. Bowyer, L. O. Hall, and W. P. Kegelmeyer, "SMOTE: Synthetic minority over-sampling technique," Journal of Artificial Intelligence Research, vol. 16, pp. 321–357, 2002.

[6] C. Elkan, "The foundations of cost-sensitive learning," in Proceedings of the 17th International Joint Conference on Artificial Intelligence (IJCAI), 2001, pp. 973–978.

[7] M. Kelly, R. Longjohn, and K. Nottingham, "The UCI machine learning repository," https://archive.ics.uci.edu.

[8] Y. Brima and M. Atemkeng, "An empirical study of machine learning robustness and scalability for imbalanced tabular clinical data in emergency and critical care," Scientific Reports, vol. 16, p. 18004, 2026.

[9] J. Hu and M. Ghelichi, "Noise immunity in in-context tabular learning: An empirical robustness analysis of TabPFN's attention mechanisms," 2026, arXiv:2604.04868.

[10] M. Saerens, P. Latinne, and C. Decaestecker, "Adjusting the outputs of a classifier to new a priori probabilities: A simple procedure," Neural Computation, vol. 14, no. 1, pp. 21–41, 2002.

[11] G. Lemaître, F. Nogueira, and C. K. Aridas, "Imbalanced-learn: A Python toolbox to tackle the curse of imbalanced datasets in machine learning," Journal of Machine Learning Research, vol. 18, no. 17, pp. 1–5, 2017.

[12] Y. Elor and H. Averbuch-Elor, "To SMOTE, or not to SMOTE?" 2022, arXiv:2201.08528.

[13] N. Natarajan, I. S. Dhillon, P. Ravikumar, and A. Tewari, "Learning with noisy labels," in Advances in Neural Information Processing Systems (NeurIPS), 2013.

[14] C. Scott, G. Blanchard, and G. Handy, "Classification with asymmetric label noise: Consistency and maximal denoising," Journal of Machine Learning Research Workshop and Conference Proceedings (COLT), vol. 30, pp. 489–511, 2013.

[15] B. Frénay and M. Verleysen, "Classification in the presence of label noise: A survey," IEEE Transactions on Neural Networks and Learning Systems, vol. 25, no. 5, pp. 845–869, 2014.

[16] H. Song, M. Kim, D. Park, Y. Shin, and J.-G. Lee, "Learning from noisy labels with deep neural networks: A survey," IEEE Transactions on Neural Networks and Learning Systems, vol. 34, no. 11, pp. 8135–8153, 2022.

[17] J. Van Hulse and T. M. Khoshgoftaar, "Knowledge discovery from imbalanced and noisy data," Data & Knowledge Engineering, vol. 68, no. 12, pp. 1513–1542, 2009.

[18] M. Koziarski, M. Woźniak, and B. Krawczyk, "Combined cleaning and resampling algorithm for multi-class imbalanced data with label noise," Knowledge-Based Systems, vol. 204, p. 106223, 2020.

[19] B. Chen, S. Xia, Z. Chen, B. Wang, and G. Wang, "RSMOTE: A self-adaptive robust SMOTE for imbalanced problems with label noise," Information Sciences, vol. 553, pp. 397–428, 2021.

[20] M. Li, H. Zhou, Q. Liu, X. Gong, and G. Wang, "WRND: A weighted oversampling framework with relative neighborhood density for imbalanced noisy classification," Expert Systems with Applications, vol. 241, p. 122593, 2024.

[21] B. Khanal, T. Dai, B. Bhattarai, and C. Linte, "Active label refinement for robust training of imbalanced medical image classification tasks in the presence of high label noise," in Medical Image Computing and Computer Assisted Intervention (MICCAI), 2024, pp. 37–47.

[22] H. Liu, M. Sheng, Z. Sun, Y. Yao, X.-S. Hua, and H.-T. Shen, "Learning with imbalanced noisy data by preventing bias in sample selection," IEEE Transactions on Multimedia, 2024, arXiv:2402.11242.

[23] G. Hu, F. Liu, M. Gong, G. Wang, and L. Peng, "Learning imbalanced data with beneficial label noise," in International Conference on Machine Learning (ICML), 2025, pp. 24 535–24 569.

[24] V. Lohweg, "Banknote authentication [dataset]," UCI Machine Learning Repository, 2012, dOI: 10.24432/C55P57.

[25] I.-C. Yeh, "Blood transfusion service center [dataset]," UCI Machine Learning Repository, 2008, dOI: 10.24432/C5GS39.

[26] U. Grömping, "South german credit data: Correcting a widely used data set," Department II, Berliner Hochschule für Technik, Tech. Rep. 4/2019, 2019.

[27] M. Hopkins, E. Reeber, G. Forman, and J. Suermondt, "Spambase [dataset]," UCI Machine Learning Repository, 1999, dOI: 10.24432/C53G6X.

[28] W. Wolberg, O. Mangasarian, N. Street, and W. Street, "Breast cancer wisconsin (diagnostic) [dataset]," UCI Machine Learning Repository, 1993, dOI: 10.24432/C5DW2B.

[29] V. Sigillito, S. Wing, L. Hutton, and K. Baker, "Ionosphere [dataset]," UCI Machine Learning Repository, 1989, dOI: 10.24432/C5W01B.

[30] S. Haberman, "Haberman's survival [dataset]," UCI Machine Learning Repository, 1999, dOI: 10.24432/C5XK51.

[31] "Statlog (heart) [dataset]," UCI Machine Learning Repository, dOI: 10.24432/C57303.

[32] R. Lyon, "Htru2 [dataset]," UCI Machine Learning Repository, 2015, dOI: 10.24432/C5DK6R.

[33] K. Mansouri, T. Ringsted, D. Ballabio, R. Todeschini, and V. Consonni, "Qsar biodegradation [dataset]," UCI Machine Learning Repository, 2013, dOI: 10.24432/C5H60M.

[34] B. Becker and R. Kohavi, "Adult [dataset]," UCI Machine Learning Repository, 1996, dOI: 10.24432/C5XW20.

[35] S. Moro, P. Rita, and P. Cortez, "Bank marketing [dataset]," UCI Machine Learning Repository, 2014, dOI: 10.24432/C5K306.

[36] R. Bock, "MAGIC gamma telescope [dataset]," UCI Machine Learning Repository, 2004, dOI: 10.24432/C52C8B.

[37] C. Sakar and Y. Kastro, "Online shoppers purchasing intention dataset [dataset]," UCI Machine Learning Repository, 2018, dOI: 10.24432/C5F88Q.

[38] "Congressional voting records [dataset]," UCI Machine Learning Repository, 1987, dOI: 10.24432/C5C01P.

[39] L. Rubini, P. Soundarapandian, and P. Eswaran, "Chronic kidney disease [dataset]," UCI Machine Learning Repository, 2015, dOI: 10.24432/C5G020.

[40] I.-C. Yeh, "Default of credit card clients [dataset]," UCI Machine Learning Repository, 2009, dOI: 10.24432/C55S3H.

[41] "Taiwanese bankruptcy prediction [dataset]," UCI Machine Learning Repository, 2020, dOI: 10.24432/C5004D.

[42] S. Tomczak, "Polish companies bankruptcy [dataset]," UCI Machine Learning Repository, 2016, dOI: 10.24432/C5F600.

[43] "Mushroom [dataset]," UCI Machine Learning Repository, 1981, dOI: 10.24432/C5959T.

[44] K. Fernandes, J. S. Cardoso, and J. Fernandes, "Cervical cancer (risk factors) [dataset]," UCI Machine Learning Repository, 2017, dOI: 10.24432/C5Z310.

[45] M. McCann and A. Johnston, "SECOM [dataset]," UCI Machine Learning Repository, 2008, dOI: 10.24432/C54305.

[46] H. A. Guvenir, B. Acar, H. Muderrisoglu, and R. Quinlan, "Arrhythmia [dataset]," UCI Machine Learning Repository, 1997, dOI: 10.24432/C5BS32.

[47] N. Kushmerick, "Internet advertisements [dataset]," UCI Machine Learning Repository, 1999, dOI: 10.24432/C5V011.

[48] M. Sikora and L. Wrobel, "Seismic-bumps [dataset]," UCI Machine Learning Repository, 2010, dOI: 10.24432/C5W902.

[49] Scania CV AB, "APS failure at scania trucks [dataset]," UCI Machine Learning Repository, 2016, dOI: 10.24432/C51S51.

[50] G. W. Brier, "Verification of forecasts expressed in terms of probability," Monthly Weather Review, vol. 78, no. 1, pp. 1–3, 1950.

[51] C. Guo, G. Pleiss, Y. Sun, and K. Q. Weinberger, "On calibration of modern neural networks," in Proceedings of the 34th International Conference on Machine Learning (ICML), 2017, pp. 1321–1330.

[52] F. Pedregosa, G. Varoquaux, A. Gramfort, V. Michel, B. Thirion, O. Grisel, M. Blondel, P. Prettenhofer, R. Weiss, V. Dubourg, J. Vanderplas, A. Passos, D. Cournapeau, M. Brucher, M. Perrot, and É. Duchesnay, "Scikit-learn: Machine learning in Python," Journal of Machine Learning Research, vol. 12, pp. 2825–2830, 2011.

[53] M. Friedman, "The use of ranks to avoid the assumption of normality implicit in the analysis of variance," Journal of the American Statistical Association, vol. 32, no. 200, pp. 675–701, 1937.

[54] F. Wilcoxon, "Individual comparisons by ranking methods," Biometrics Bulletin, vol. 1, no. 6, pp. 80–83, 1945.

[55] S. Holm, "A simple sequentially rejective multiple test procedure," Scandinavian Journal of Statistics, vol. 6, no. 2, pp. 65–70, 1979.

[56] J. Demšar, "Statistical comparisons of classifiers over multiple data sets," Journal of Machine Learning Research, vol. 7, no. 1, pp. 1–30, 2006.